Day 116: Stacking and Blending From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 5 | Complete study guide

Watch: 7.7 Stacking (L07: Ensemble Methods) (Sebastian Raschka) | Stacking Explained | Advanced Ensemble Learning in Machine Learning (Tech Pulse Labs)

Script: `day116_stacking_blending.py` · every number in this guide is copied by program from `day116_run_output.txt`, the real output of that script.

Stacking trains a second model to combine the predictions of several first-level models. It is written from scratch here, with five base models from earlier lessons, and tested honestly against the obvious alternative: pick the single best model by cross-validation. The result is modest. Stacking was level with that choice on breast cancer (+0.0026) and Titanic (-0.0017), and worse on digits (-0.0082, on all 5 splits); a plain average of the models did at least as well every time. Stacking needs models that make different mistakes, and on Titanic 0.1073 of test rows are wrong for every model.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

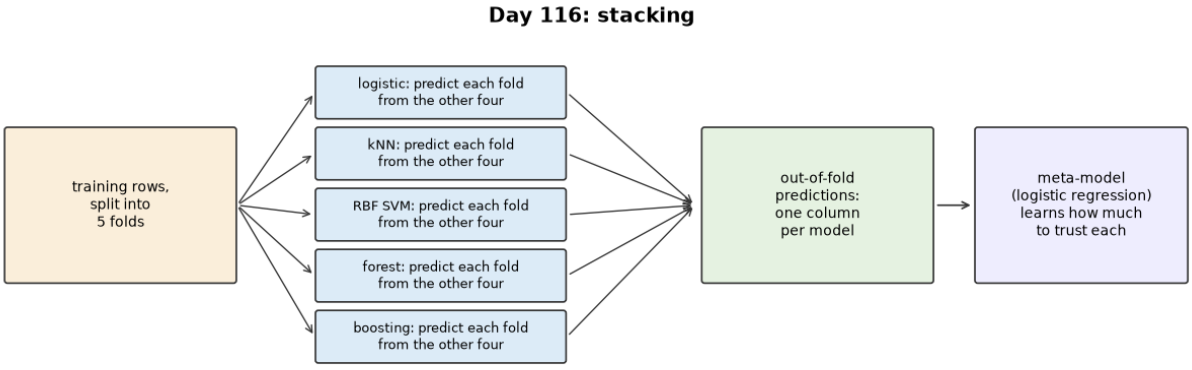
An **ensemble** combines several models. **Stacking** trains a **meta-model** on the base models' predictions. Those predictions must be **out-of-fold**: each training row predicted by a copy of the model that never saw it, exactly as test rows will be. **Blending** is the simpler version: hold back part of the training rows and train the meta-model on predictions for those.

Here the meta-model is softmax regression (logistic regression for any number of classes), and its inputs are the base models' log-odds.

REAL-WORLD SCENARIO

A hospital asks five specialists for a diagnosis and learns, from past cases, how much to trust each one. The trust must come from cases the specialists had not already been told the answer to.

SYSTEM DIAGRAM



Each base model predicts every training row from a fold it never saw; the meta-model learns from those predictions.

Today's part for the week's build

`day116_exercise.py` is softmax regression, out-of-fold predictions and the stack, checked on a useful and a useless model. Its real output:

ACTUAL OUTPUT

```
out-of-fold: rows a fold's model saw that it then predicted: 0
meta-weights (class 1 against class 0): useful model +2.223, useless model +0.000 (its log-odds
    are always 0)
stacked accuracy on its training rows: 0.960; probabilities sum to 1: True
```

Base models, from earlier lessons: softmax regression, kNN (Day 115), an RBF SVM (Days 112-113), a random forest (Day 107) and gradient boosting (Day 109, two classes only). Data: breast cancer, Titanic (the repository's own copy) and digits.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: two models, one meta-model

DEFINITION — start here, no prior knowledge assumed

Eight rows and two models that are each wrong on different rows. Their average is right on all eight; the meta-model learns a weight for each.

REAL-WORLD SCENARIO

Two witnesses who each misremember different details: together they tell the whole story.

MATHEMATICS

$$features = \log(p / (1 - p)) \text{ for each base model;} \quad P(class) = softmax(features \cdot W + b)$$

IMPLEMENTATION

```
def softmax(Z):
    Z = Z - Z.max(1, keepdims=True); E = np.exp(Z); return E / E.sum(1, keepdims=True)
def fit_softmax(F, y, k, lam=1e-3, steps=500, lr=0.5):
    W = np.zeros((F.shape[1], k)); b = np.zeros(k); Y = np.eye(k)[y]
    m = [np.zeros_like(W), np.zeros_like(b)]; v = [np.zeros_like(W), np.zeros_like(b)]
    for t in range(1, steps + 1):
        G = (softmax(F @ W + b) - Y) / len(y)
        for i, (p, g) in enumerate(((W, F.T @ G + lam * W), (b, G.sum(0)))):
            m[i] = 0.9 * m[i] + 0.1 * g; v[i] = 0.999 * v[i] + 0.001 * g * g
            p -= lr * (m[i] / (1 - 0.9 ** t)) / (np.sqrt(v[i] / (1 - 0.999 ** t)) + 1e-8)
    return W, b
```

ACTUAL OUTPUT

```
model A right on 0.750 of rows, model B on 0.875, their average on 1.000
the meta-model weighs A's log-odds by 3.247 and B's by 3.165; it gets 1.000 of these rows right
A is wrong on rows [3, 8], B on rows [4]: never the same row, so a mix can fix both (on eight
    rows, the meta-model is scored on the rows it fitted)
Actual output from this lesson's run.
```

What it means

A is right on 0.750 of rows and B on 0.875; their mistakes never fall on the same row, so a mix gets 1.000. The meta-model gives both similar weights (3.247 and 3.165). On eight rows it is scored on the rows it fitted, so this only shows the mechanism; section 3 asks whether it helps on held-out rows.

WHY CHOOSE THIS?

Shows why combining can beat both parts: different mistakes.

WHY NOT THIS? (TRADEOFFS)

Tiny and in-sample; real gains are far smaller.

2. The leakage trap

DEFINITION — start here, no prior knowledge assumed

If the meta-model learns from the base models' predictions on their *own* training rows, it sees models that look better than they are, and trusts the ones that memorise most. Out-of-fold predictions avoid this.

REAL-WORLD SCENARIO

Ranking students by how well they answer questions they have already seen the answers to.

IMPLEMENTATION

```
def oof(fit, X, y, k, seed):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), FOLDS); P =
        np.zeros((len(y), k))
    for i in range(FOLDS):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(FOLDS) if j != i])
        P[va] = fit(X[tr], y[tr], k, seed)(X[va])
    return P
def stack(names, X, y, Xte, k, seed, leak=False):
    fits = {n: BASE[n](X, y, k, seed) for n in names}
    tr = [features(fits[n](X) if leak else oof(BASE[n], X, y, k, seed), k) for n in names]
    te = [features(fits[n](Xte), k) for n in names]
    W, b = fit_softmax(np.hstack(tr), y, k)
    return softmax(np.hstack(te) @ W + b), W, {n: fits[n](Xte) for n in names}
```

ACTUAL OUTPUT

```
10 splits per dataset; the same base models, the meta-model trained on two kinds of predictions
Breast cancer: each base model's accuracy on its OWN training rows: logistic 0.9887, kNN 0.9638, RBF SVM
    0.9834, forest 1.0000, boosting 1.0000
  out-of-fold : test accuracy 0.9819; meta-weights logistic +0.31, kNN -0.03, RBF SVM +2.30, forest +0.05,
      boosting +0.09
  in-sample   : test accuracy 0.9830; meta-weights logistic +2.12, kNN +2.41, RBF SVM -0.04, forest +2.57,
      boosting +2.83
  out-of-fold minus in-sample: -0.0012; better on 3, worse on 3 of 10
Titanic: each base model's accuracy on its OWN training rows: logistic 0.7995, kNN 0.8262, RBF SVM 0.8371,
    forest 0.9839, boosting 0.9244
  out-of-fold : test accuracy 0.8209; meta-weights logistic +0.46, kNN +0.24, RBF SVM +0.31, forest +0.28,
      boosting +0.53
  in-sample   : test accuracy 0.7937; meta-weights logistic -0.49, kNN -0.26, RBF SVM -0.21, forest +3.26,
      boosting +0.29
  out-of-fold minus in-sample: +0.0272; better on 9, worse on 1 of 10
Actual output from this lesson's run.
```

What it means

On breast cancer the trap did no measurable harm (-0.0012, better on 3 and worse on 3 splits), although the meta-model learned very different weights: out-of-fold it trusts the SVM (+2.30); in-sample it spreads its trust over the other four (boosting +2.83) and ignores the SVM, while the forest and boosting score 1.0000 on their own training rows. Every model is good on breast cancer, so trusting the wrong one costs little.

Titanic shows the trap. The forest is right on 0.9839 of its own training rows but on 0.8007 of new ones (section 3), and the in-sample meta-model puts nearly all its weight on it (+3.26, with negative weights on three other models). Its test accuracy falls to 0.7937 against 0.8209 out-of-fold: +0.0272 in favour of out-of-fold predictions, better on 9 of 10 splits. Here the damage came where one base model memorised far more than it generalised.

WHY CHOOSE THIS?

Out-of-fold predictions are the honest input, whatever the effect on accuracy.

WHY NOT THIS? (TRADEOFFS)

It costs five extra fits per base model; on easy data the leak may not show at all.

3. Is it better?

DEFINITION — start here, no prior knowledge assumed

Every base model, the single model with the best out-of-fold accuracy (what you would pick without stacking), a simple average, blending and stacking, on the same splits.

REAL-WORLD SCENARIO

Five weather forecasts: trust the one with the best record, average them, or learn a weighting.

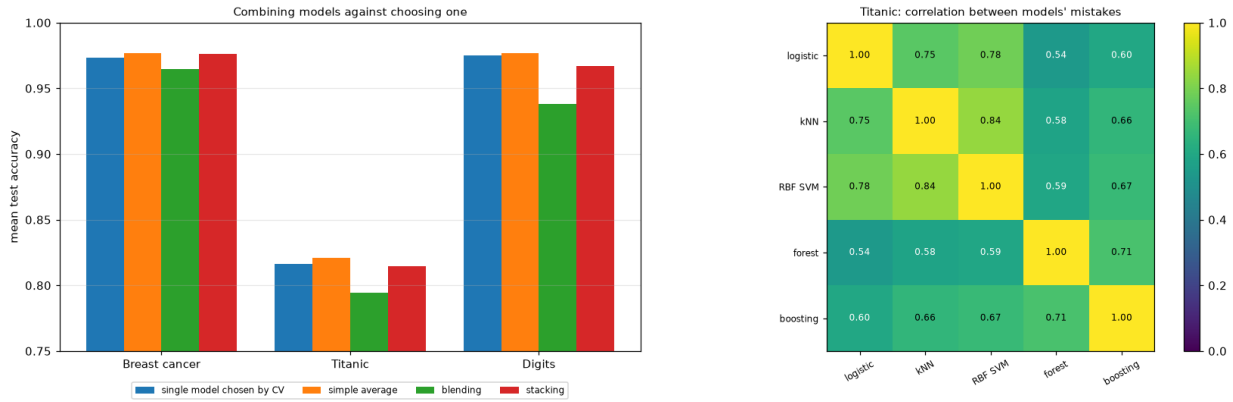
IMPLEMENTATION

```
def blend(names, X, y, Xte, k, seed, frac=0.3):
    rng = np.random.RandomState(seed); o = rng.permutation(len(y)); h = int(len(y) * frac); ho,
        fit_rows = o[:h], o[h:]
    fits = {n: BASE[n](X[fit_rows], y[fit_rows], k, seed) for n in names}
    W, b = fit_softmax(np.hstack([features(fits[n](X[ho]), k) for n in names]), y[ho], k)
    return softmax(np.hstack([features(fits[n](Xte), k) for n in names]) @ W + b)
```

ACTUAL OUTPUT

```
stacking: meta-model on 5-fold out-of-fold predictions; blending: base models on 70% of the training rows,
    meta-model on the other 30%
'chosen by CV' is the single base model with the best out-of-fold accuracy: what you would pick without
    stacking
Breast cancer: 569 rows, 2 classes, 20 splits
  logistic                    0.9725  (std 0.0126)
  kNN                         0.9588  (std 0.0160)
  RBF SVM                     0.9769  (std 0.0113)
  forest                      0.9596  (std 0.0148)
  boosting                    0.9596  (std 0.0179)
  single model chosen by CV   0.9734  (std 0.0129)
  simple average              0.9766  (std 0.0135)
  blending                    0.9646  (std 0.0146)
  stacking                    0.9760  (std 0.0123)
  stacking minus the CV choice: +0.0026 [-0.0020, +0.0070]; better on 10, worse on 6 of 20
  average minus the CV choice: +0.0032 [-0.0023, +0.0091]; better on 11, worse on 7 of 20
  stacking took 4.7 s per split (median): every base model is trained 6 times
Titanic: 891 rows, 2 classes, 20 splits
  logistic                    0.7860  (std 0.0222)
  kNN                         0.8015  (std 0.0218)
  RBF SVM                     0.8192  (std 0.0194)
  forest                      0.8007  (std 0.0187)
  boosting                    0.8166  (std 0.0133)
```

```
  single model chosen by CV   0.8164  (std 0.0201)
  simple average              0.8209  (std 0.0161)
  blending                    0.7944  (std 0.0527)
  stacking                    0.8147  (std 0.0251)
  stacking minus the CV choice: -0.0017 [-0.0131, +0.0069]; better on 11, worse on 6 of 20
  average minus the CV choice: +0.0045 [-0.0009, +0.0095]; better on 11, worse on 6 of 20
  stacking took 5.4 s per split (median): every base model is trained 6 times
Digits: 1797 rows, 10 classes, 5 splits
  logistic                    0.9677  (std 0.0057)
  kNN                         0.9584  (std 0.0119)
  RBF SVM                     0.9759  (std 0.0059)
  forest                      0.9725  (std 0.0069)
  single model chosen by CV   0.9751  (std 0.0056)
  simple average              0.9766  (std 0.0064)
  blending                    0.9380  (std 0.0075)
  stacking                    0.9670  (std 0.0034)
  stacking minus the CV choice: -0.0082 [-0.0100, -0.0059]; better on 0, worse on 5 of 5
  average minus the CV choice: +0.0015 [-0.0026, +0.0056]; better on 2, worse on 1 of 5
  stacking took 20.1 s per split (median): every base model is trained 6 times
the splits share rows, so these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```



Left: accuracy of each approach on the three datasets. Right: how often the models on Titanic are wrong on the same rows.

What it means

Stacking did not beat the single model chosen by CV: +0.0026 on breast cancer and -0.0017 on Titanic, both with intervals spanning zero, and -0.0082 on digits, worse on 5 of 5 splits. The simple average did at least as well as stacking everywhere (0.9766 on breast cancer, 0.8209 on Titanic, 0.9766 on digits), with nothing to learn. Blending was worst (0.7944 on Titanic, with a large spread, 0.0527): its base models lose 30% of their training rows, and its meta-model learns from few rows. On digits the meta-model must learn 40 x 10 weights from out-of-fold log-probabilities, which may explain why it falls behind; that was not tested further.

WHY CHOOSE THIS?

A fair test: same splits, and against the choice you would make anyway.

WHY NOT THIS? (TRADEOFFS)

Stacking trains every base model six times: once per fold, and once on all the rows.

4. What the meta-model learned, and how different the mistakes are

DEFINITION — start here, no prior knowledge assumed

The meta-model's weights on Titanic, and the correlation between the models' mistakes: 1 means two models are wrong on exactly the same rows.

REAL-WORLD SCENARIO

A panel of experts who all trained at the same school tend to make the same errors.

ACTUAL OUTPUT

```
Titanic, 20 splits: mean meta-weight on each model's log-odds: logistic +0.356, kNN +0.176, RBF SVM +0.298,
    forest +0.263, boosting +0.408
correlation between models' mistakes (1 = the same rows wrong):
              logistic       kNN   RBF SVM    forest  boosting
  logistic       1.000     0.745     0.782     0.540     0.599
  kNN            0.745     1.000     0.843     0.580     0.660
  RBF SVM        0.782     0.843     1.000     0.592     0.673
  forest         0.540     0.580     0.592     1.000     0.711
  boosting       0.599     0.660     0.673     0.711     1.000
test rows every model gets wrong: 0.1073; rows at least one gets right and at least one wrong: 0.2144
no combination of these models can fix the first kind; the second kind is all stacking has to work with
Actual output from this lesson's run.
```

What it means

The weights are spread across all five models, boosting highest (+0.408). The mistakes overlap strongly: kNN and the SVM correlate at 0.843, the forest and logistic regression least (0.540). On 0.1073 of test rows every model is wrong, so no combination can help there; only 0.2144 of rows are disputed, and that is all stacking has to work with.

WHY CHOOSE THIS?

Explains the small gains: the room for improvement is measured.

WHY NOT THIS? (TRADEOFFS)

Correlation of 0/1 errors is a rough measure of diversity.

5. Diversity

DEFINITION — start here, no prior knowledge assumed

Stacking three forests that differ only in their seed, against stacking three different kinds of model.

REAL-WORLD SCENARIO

Asking the same person three times, or three different people.

ACTUAL OUTPUT

```
breast cancer, 10 splits
  three forests (different seeds): stack 0.9602, best member on each split 0.9643
  forest, logistic, kNN:           stack 0.9789, best member on each split 0.9789
('best member on each split' is chosen by looking at the test rows, so it flatters the members)
Actual output from this lesson's run.
```

What it means

Three forests stacked reach 0.9602, below the best of the three (0.9643): copies of one model add nothing. Three different models stacked reach 0.9789, equal to their best member on each split (0.9789), and that best member was picked by looking at the test rows, which no real procedure can do. Diversity is what makes stacking worth trying.

WHY CHOOSE THIS?

Shows which ensembles are worth building.

WHY NOT THIS? (TRADEOFFS)

Ten splits of one dataset.

Interview preparation — technical questions

Q: What is stacking?

A: A meta-model trained on base models' out-of-fold predictions.

Q: Why out-of-fold predictions?

A: In-sample predictions make memorising models look perfect (forest and boosting 1.0000 on their own training rows).

Q: Stacking or averaging?

A: Try the average first: here it matched or beat stacking on all three datasets.

Q: Stacking or blending?

A: Stacking uses every row for both levels; blending lost 30% of rows and scored 0.7944 on Titanic.

Q: How do you pick base models?

A: Different kinds, with different mistakes; three forests stacked gained nothing.

Interview preparation — theoretical questions

Q: Why can an ensemble beat its members?

A: When their mistakes differ: on eight rows, two 0.750 and 0.875 models combined to 1.000.

Q: What limits stacking?

A: Rows every model gets wrong: 0.1073 on Titanic.

Q: Why use log-odds as meta-features?

A: They are on the scale logistic regression combines linearly.

Q: Why might stacking lose to the best single model?

A: The meta-model adds weights to estimate; with similar, correlated models it mostly adds noise.

Key takeaway and what's next

KEY TAKEAWAY

Stacking is a clean idea: learn how much to trust each model, from predictions it made on rows it never saw. On these three datasets it did not beat simply picking the best model, and a plain average did as well or better. Its gains depend on diverse models with different mistakes, and these models agree too often for a meta-model to find much.

- Stacking minus the CV choice: +0.0026, -0.0017, -0.0082 (breast cancer, Titanic, digits).
- Simple average: 0.9766, 0.8209, 0.9766.
- Titanic: 0.1073 of rows wrong for every model; 0.2144 disputed.
- Three forests stacked: 0.9602, below their best member.

Suggested watch order, if starting from zero

- 1. 7.7 Stacking (L07: Ensemble Methods) (Sebastian Raschka) — a lecture on stacking and out-of-fold predictions.
- 2. Stacking Explained | Advanced Ensemble Learning in Machine Learning (Tech Pulse Labs) — stacking explained, with a code walkthrough.

Day 117 preview

- **Imbalanced data**: when one class is rare, accuracy misleads; SMOTE and class weights compared on real rare-class problems.